Ripple: Test, Debug and Fix Plan

Bug Bounty / Main / Redundant branches · Team HitNRun: Chirantan, Nandini, John · 15 Oct 2026

Goal. Find, analyse and fix every error, unwanted thing and weak spot in Ripple, on all three branches (`main`, `bugbounty`, `redundant`), so the product is always working, secure, honest and clean. Three people, three tracks, one shared checklist, one shared bug ledger.
Includes the second master prompt: Postman collection and CLI runs, Playwright on every control, security, fixes and a final report (section 6.5, section 7.4, Appendix A).
What you send back. Each of you returns (1) your fixes as a pushed branch or patch, (2) your report in the template in section 11, (3) your bug ledger, (4) your control matrix. Chirantan pastes the three reports into the working chat to continue development.

1. How to use this document

1. Read sections 2 to 5 together (15 minutes), agree who owns what, then work only in your track. Cross-review is in section 4.
2. Section 6 is the master checklist: every item from the notebook photos, the long Claude Code audit prompt, and about 30 things that were missing. Tick the box only when there is evidence (a test, a screenshot, a command output).
3. Every bug you find goes in the ledger (section 9) the moment you find it, even if you fix it five minutes later.
4. Nothing is "done" without a regression test or a written reason why a test is impossible.

2. Ground rules

Safety

- Test only our own app, locally or on our own deployment. Never other teams' systems.
- Use the demo business and synthetic data. No real customers, no real money, no real WhatsApp or external messages.
- No load tests against the deployed site. No denial-of-service attempts.
- Never paste keys, passwords, tokens or session cookies into reports, chats or screenshots. Write `[REDACTED]`.
- Never weaken a check, skip a test, or delete a failing test to get green. A failing test is a finding.
- Check `git status` before you start. Do not discard anyone's uncommitted work. No `git reset --hard`, no force-push.

Branches (the repo rules from the README apply)

- **Attack phase 12:00 to 2:00 AM:** `bugbounty` is frozen. No commits to it, at all.
- **Fixing phase 2:00 to 3:30 AM:** create `fixing` from `bugbounty`, commit fixes there only, nothing after 3:30 AM. Merge into `main` afterwards.
- Keep `main` and `bugbounty` in sync until the attack phase starts. Never delete `bugbounty` or `fixing`.
- **Our working protocol:** fix on a short branch `fix/<initials>-<bug-id>` cut from `main`; one bug, one commit; run the tests; open a pull request to `main`. Only **Chirantan** moves `main` into `bugbounty` and `redundant` (fast-forward), and only outside the freeze.
- If the bug-bounty round also has us attack other teams: at most 10 reports overall, 4 per target team, clear and reproducible; critical exposures go privately to the organisers.

3. Where we stand today (verified facts, 15 Oct)

Area	What we know

Stack	FastAPI + SQLite in `backend/app`; React 18 + Vite + TypeScript in `frontend/src`; one Playwright smoke script in `e2e/`; Docker + `railway.json` for deployment.
Repo shape (notebook p.1)	**7 folders:** backend, docs, e2e, fixtures, frontend, scripts, tests. **11 root files:** .dockerignore, .env.example, .gitattributes, .gitignore, CLAUDE.md, Dockerfile, README.md, khata-csv-export.diff, railway.json, requirements-voice.txt, requirements.txt. Every folder and every file needs a verdict: keep, fix, merge or delete.
Branches	After the last push, `main`, `bugbounty` and `redundant` were all at commit `4a8d820b`. Re-check with `git log -1` before you start; others may have pushed since.
Mounted API routes	73 handlers in `app_routes.py` (66) and `auth_routes.py` (7): 26 GET, 32 POST, 7 PATCH, 4 PUT, 4 DELETE. Plus `/api/health`, `/health` and a catch-all that serves the web app. API docs are switched off.
Suspect: code that is not mounted	`backend/app/api/routes.py` (13 routes, including `/demo/reset` and `/parties/upload`), `store.py`, `forecast.py`, `importer.py`, `models.py`, `config.py`, `assistant/chat.py`, `assistant/speech.py` (about 1,700 lines) and `frontend/src/services/api.ts`. `main.py` mounts only the two routers above, so this looks like a parallel, abandoned backend. **Verify first**; if confirmed, it is "unwanted" (notebook Q4/Q5): delete or properly integrate, but do not leave it half-live.
Tests	127 backend tests pass (1 skipped). 59 frontend unit tests pass, but only in a copy of the project at a path without `#`: Vite and Vitest break inside `M# 2026`. Do all frontend builds from a clean path such as `C:\ripple`.
Already in place (verify, do not assume)	CSRF header `X-Ripple` on writes; 413 on large bodies; `nosniff`, `X-Frame-Options: DENY`, Referrer-Policy, Permissions-Policy, a Content-Security-Policy on pages; no-store on the API; session cookie `HttpOnly` + `SameSite=Lax` + Secure behind HTTPS; rate limits on login and AI routes; scrypt password hashing; role checks (owner, accountant, supervisor); CORS only when `ALLOWED_ORIGINS` is set; every record scoped by `business_id`.
Gaps visible at a glance (confirm each)	No privacy-policy or terms page; no cookie banner; no `robots.txt`, sitemap or social-preview image; one static page `<title>`; no HSTS header; no custom page for unknown routes; no analytics; no dependency audit on record; demo credentials printed in the README; an environment override (`RIPPLE_TODAY`) that pins "today" and must never be set in production; stray patch files (`khata-csv-export.diff`, `docs/optional-ui-patch.diff`).

4. Who does what

Track	Owner	Scope (you own the testing and the fixes here)
A Product and intelligence	**Chirantan** also release lead	Playwright: every control on **Today**, **Engine** (Network, Flow, Decisions, Timeline play/pause/step, what-if bar, scenario banner, Inspector, Table), **Book** (Position, Risk & Finance incl. payment check and flag buttons, Ledger), notification bell, command line (`/`), Munim chat, voice input and read-aloud in all 10 languages, keyboard shortcuts. Also: hardcoded-versus-real analysis (notebook Q1 to Q3), AI and Munim failure modes, engine and risk correctness, the branch sync, merging, and assembling the final report.
B Screens and data	**Nandini**	Playwright: every control on **Login**, **Setup** (describe the business, review, approve), **Khata** (party drawer, reminders, composer, datasheet download), **Orders** (sales stages and Undo, purchase orders, "Can I take this order?"), **Inventory** (materials, recipes, movements, filters), **Settings** (every section), payment dialog. Also: all form validation, CSV import and export, language and number/date formats, mobile and tablet layouts, colour contrast, keyboard and screen-reader use, and the website basics from the notebook (meta tags, favicon, social preview, sitemap, robots, 404 page, images, load speed, legal pages, cookie notice, broken links, call to action).
C Backend and security	**John**	The complete endpoint inventory (static, dynamic, wanted, unwanted) and the **Postman collection and CLI runs** (section 7.4); the not-mounted code question; authentication, authorisation, cross-business access, mass assignment, rate limiting, CSRF, CORS, headers, cookies, HTTPS; injection, XSS and uploads; secrets and git-history scan; dependency audit; production settings, Docker, Railway, health checks; logging; database integrity, migrations and backup; concurrency; API fuzzing; the backend test suite.

Cross-review (each fix is checked by a second person)

Fixes by	Are reviewed and re-tested by	Why this pairing
Chirantan (A)	Nandini	Fresh eyes on engine, risk and AI behaviour as a user.
Nandini (B)	John	Security view of every form, upload and export she touches.
John (C)	Chirantan	Confirm security fixes did not break any screen or the demo flow.

Shared by all three

- **Kickoff (15 min):** confirm the branch protocol, set up a clean-path copy, run the baseline (section 10).
- **Every 60 minutes:** 5-minute sync: new bugs, conflicts, what is merged. Chirantan merges, everyone pulls `main`.
- **Use the master prompt:** each of you can run the Claude Code master prompt for your own track (Appendix A). It finds, fixes and retests; you still review the diff.
- **Own the regression tests:** whoever fixes a bug writes its test.
- **Think like an attacker (bug bounty):** other teams will try login, IDOR, XSS, injection, rate limits, error leakage. Each track also asks "how would a stranger break this?".

5. Timeline (adjust to the real deadline)

When	Phase	Output
T+0:00	Kickoff and baseline	Branch protocol agreed. Clean-path copy. Baseline test results recorded (what already fails).

T+0:15 to 1:30	Inventory	A: control list for A's screens. B: control list for B's screens. C: endpoint inventory (`docs/endpoint-inventory.json`), orphan-code verdict, secrets scan.
T+1:30 to 5:00	Test and fix loop	Rounds of 60 minutes: test, log bugs, fix the P0/P1 first, regression test, pull request, sync. Control matrix filled as you go.
T+5:00 to 6:00	Cross-review	Each person re-tests a colleague's fixes and the full demo flow. Full test suites run on the merged code.
T+6:00 to 7:00	Report and release	Reports and ledgers sent to Chirantan. Final merge to `main`, then sync to `bugbounty` and `redundant` outside the freeze.

6. Master checklist

Owner letters: A Chirantan, B Nandini, C John. "Evidence" means a test name, screenshot, command output or ledger ID. Items from the notebook photo, page 2, are numbered N1 to N40 as read from the handwriting. A few words at the right edge of the photo are cut off; confirm them against the notebook.

6.1 Notebook page 1: "Bug fix" (repo shape and the six questions)

#	Item	What to do and what to hand in	Owner	✓
Q0	7 folders + files inside; 11 random files	For each of the 7 folders and 11 root files write one line: purpose, used or not, keep / fix / merge / delete. Include the stray `.diff` files, `CLAUDE.md`, `requirements-voice.txt` and the duplicate API modules.	C + A	☐
Q1	What share of the app is hardcoded, and what is dynamic (user input, calculated output)?	For every page, list each displayed value and class it: **user input**, **stored record**, **engine-calculated**, **hardcoded constant**, **static text**. Report the share per page. "Vibe-coded" cannot be measured objectively, so we use the hardcoded and static share as the honest measure.	A	☐
Q2	List the hardcoded values, pages, and everything hardcoded and static	Use the starter list in section 8, extend it by grep (`BP =`, `DAYS =`, numeric literals in `services/`, `seed.py`, message templates, i18n files, demo accounts). Mark each: should be a setting, should stay a constant, or is demo-only.	A + B	☐
Q3	How much can we change into something real, practical and meaningful	For each hardcoded item decide: make it a setting, derive it from the books, import real data, or label it clearly as an assumption. Give an effort estimate (S/M/L). Do the S ones in this round.	A	☐
Q4	How many endpoints in total: static, dynamic, wanted, unwanted	Produce `docs/endpoint-inventory.json`. Static = health, web-app files, catch-all. Dynamic = data routes. Wanted = called by the frontend or documented. Unwanted = orphan, duplicate, debug, unmounted. Re-count on `main`: our local count is 73 + 3.	C	☐
Q5	What can be reduced, optimised, modified	Candidates: unmounted code (~1,700 lines), the 270 kB Engine chunk and 299 kB main chunk, `useWrite` refetching every query after any write, polling of risk and fix chances, heavy `/fixes` evaluation, duplicated screens, unused i18n keys, unused routes. Measure before and after.	C + B	☐
Q6	Website security: rate limiting, authorisation, authentication (line continues on page 2)	Covered in full by 6.2 (N20 to N40) and section 7.	C	☐

6.2 Notebook page 2: the 40 checkpoints

#	Checkpoint	How we check it in Ripple	Owner	✓
N1	Privacy policy page	Add `/privacy`: what we store (business records, session cookie), AI provider disclosure (text goes to the AI service), voice (browser speech services), retention (sandbox deleted after 24 h), contact. Link from login and footer. Indian DPDP Act notice wording to be checked by the team.	B	☐
N2	Terms and Conditions page	Add `/terms`: demo/synthetic data notice, forecasts are estimates not advice, "not financial advice" on offers, acceptable use. Link beside the privacy page.	B	☐
N3	Take secrets off the frontend	Search the built bundle in `frontend/dist` and source for keys, tokens, passwords, internal URLs. Only `VITE_*` public values may appear. `GROQ_API_KEY` must exist only on the server.	C	☐

N4	Force HTTPS	Deployed site: HTTP must redirect to HTTPS; add `Strict-Transport-Security` when behind HTTPS; confirm the cookie turns `Secure` (`COOKIE_SECURE=auto`).	C	☐
N5	Cookie consent banner	Decide: the session cookie is strictly necessary, so a banner is only needed if we add analytics or other non-essential cookies. If analytics (N21) is added, add consent first. Otherwise add a short cookie notice linking to the privacy page.	B	☐
N6	Meta titles and descriptions	Today every page says "Ripple". Set a title per route (Today, Engine, Book...) and a description for public pages (login, privacy, terms).	B	☐
N7	Social preview image	Add `og:title`, `og:description`, `og:image`, `twitter:card`; a 1200x630 image of the Flow or Today screen.	B	☐
N8	Favicon	Present as `favicon.svg`. Check it loads on every route (a relative path was fixed once), add PNG and Apple-touch versions.	B	☐
N9	Sitemap	Only public pages belong in it (login, privacy, terms). The signed-in app must not be listed.	B	☐
N10	robots.txt	Allow public pages; disallow `/app/` and `/api/`. Served as a real file, not the web-app fallback.	B	☐
N11	Load speed	Run Lighthouse on login and Today (mobile profile). Targets: LCP under 2.5 s, no layout shift. Look at bundle split, font loading, first API calls.	B	☐
N12	Alt text on images	Every `img`, SVG icon with meaning and chart has a name or is marked decorative (`aria-hidden`). The futures chart and runway chart need text equivalents.	B	☐
N13	Compress images	Social image and any screenshots under 200 kB; SVG minified.	B	☐
N14	"Check page ..." (line cut off)	Read as: check page speed / page titles. Covered by N6 and N11. Confirm against the notebook.	B	☐
N15	Custom 404 page	Unknown route inside `/app` and outside it shows a helpful page with a way home; the API keeps returning JSON 404 for unknown `/api` paths.	B	☐
N16	Fix colour contrast	WCAG AA (4.5:1 text, 3:1 large text and icons) in Night, Light and high-contrast themes. Pay attention to dim grey text, haldi on dark, status chips, chart labels.	B	☐
N17	Make it mobile friendly	360, 390, 768 px widths; no horizontal scroll; tap targets 44 px; tables wrap; the bottom navigation does not hide content.	B	☐
N18	Fix broken links	Crawl every `href` and `navigate()` target, including links built in code (alerts, Today actions, Munim links, `?tab=` links). External links: `rel="noopener noreferrer"`.	B	☐
N19	Form validation	Every form: empty, too long, wrong type, negative, decimals, special characters, double submit, server error shown correctly. Client and server rules must agree.	B	☐
N20	Spam protection	Login, demo creation (`/api/auth/demo` makes a new business each call: limit it), team invites, AI chat, setup draft. Rate limits, body limits, optional CAPTCHA decision.	C	☐
N21	Set up analytics	Decide yes or no. If yes: privacy-friendly, no business data in events, consent before loading (N5). If no, record the decision.	A	☐
N22	One clear call to action	Login: "Open demo business" is obvious. Each screen has one primary action (Today: first action; Book: Check a payment; Orders: new order).	A	☐
N23	Hide API keys	No key in the repo, bundle, error messages, logs, screenshots or the AI prompt. The "missing key" message must not reveal key names to end users beyond what is intended.	C	☐

N24	Check env variables	Compare `.env.example`, code (`os.getenv`), Dockerfile and Railway settings. Required vs optional; safe defaults; the app refuses to start in production with a missing or default secret; `RIPPLE_TODAY` unset.	C	☐
N25	Check keys in git	Scan the whole history of all branches (not only the latest commit) with `gitleaks` or `trufflehog`. If a key ever existed, rotate it. Check `.gitignore` covers `.env` and database files.	C	☐
N26	Protect admin routes	Owner-only routes (`/api/team`, `/api/data/reset`, `/api/data/load-demo`, settings, export) return 401 without a session and 403 for accountant and supervisor, tested at the API, not just hidden in the UI.	C	☐
N27	Add auth	Every non-public route needs a session. List the public ones on purpose: login, demo, health, static files, privacy, terms.	C	☐
N28	Check user permissions	Three roles x every route matrix. Supervisor must never receive money fields (the server strips amounts). Accountant cannot manage the team.	C	☐
N29	Sanitise user input	Every text field (party names, notes, descriptions, chat, setup text, labels): length, control characters, HTML, CSV formulas, Unicode edge cases.	C	☐
N30	Protect against XSS	Put `<img src=x onerror=...>` and `<script>` into every stored field; check every place it renders, including Munim replies, toasts, the Flow nodes, CSV, WhatsApp message drafts and print views. No `dangerouslySetInnerHTML` without a sanitiser.	C	☐
N31	Protect against SQL injection	Confirm every query is parameterised (grep for f-strings and `+` in SQL; the table-name f-strings in `db.py` must only take constants). Try `' OR 1=1 --` in every filter, search and ID.	C	☐
N32	Check database rules	Foreign keys, NOT NULL and CHECK constraints, unique numbers (invoice, order), integer paise only, migration safety on an old database file, WAL mode and locking, backup and restore of the `/data` volume.	C	☐
N33	Secure file uploads	Find every upload or import (setup text, receipts, CSV/import code, `parties/upload` in the unmounted module). Type, size, name, content checks; no execution; stored outside the web root.	C	☐
N34	CSRF protection	Writes without `X-Ripple: 1` are rejected; check form posts, logout, demo creation and cross-origin attempts from a second origin.	C	☐
N35	Check CORS settings	Default is no CORS. With `ALLOWED_ORIGINS` set, only those origins; never `*` with credentials; preflight behaviour correct.	C	☐
N36	Add security headers	Present: nosniff, frame deny, referrer, permissions, CSP on pages. Add HSTS (N4). Check the CSP does not break fonts, charts or voice, and consider tightening `style-src 'unsafe-inline'`.	C	☐
N37	Secure cookies	Session cookie: HttpOnly, SameSite, Secure in production, sensible lifetime, cleared on logout, invalid after "log out everywhere". No tokens in URLs or local storage.	C	☐
N38	Check prod settings	Debug off, docs off (already), error details hidden, strong `DEMO_PASSWORD`, no `RIPPLE_TODAY`, persistent volume, health route, workers, logs. Compare local and deployed behaviour.	C	☐
N39	Authentication (end of Q6 line)	Login, logout, expiry, lockout and rate limit, password rules, account enumeration, session list and revocation. See section 7.	C	☐
N40	"Check ..." at the end of the page (cut off)	Read as "check permissions / check prod settings". Covered by N28 and N38. Confirm against the notebook.	C	☐

6.3 From the long Claude Code audit prompt: sections 1 to 10

This is the first audit prompt. It ended in the middle of section 10, and a second, complete master prompt followed (Postman, Playwright, security, fixes). Section 6.5 maps all 12 phases of the second prompt, and the rest of this plan fills in what the first one never reached.

Prompt §	Requirement	Where it lands in this plan	Owner	✓
1	Safety and change control	Section 2. Also: baseline recorded before any change; test database and synthetic users only; mock external side effects.	All	☐
2	Deliverables	`docs/production-readiness-audit.md`, `docs/bug-ledger.md`, `docs/endpoint-inventory.json`, `docs/ui-control-coverage.json`, `postman/collections/ripple-api.json`, Playwright tests, new unit and integration tests. Each person writes to `docs/audit/<name>/`; nobody overwrites another's file. Chirantan assembles.	A	☐
3	Inventory the whole application	Frontend: every route, nav item, button, card, row, link, menu, tab, toggle, form, dialog, drawer, popover, search/filter/sort, upload, download, voice control, shortcut, conditional control. Backend: every route with method, auth, schema, tables touched, frontend caller, test status. Compare the two lists in both directions (frontend calling a route that does not exist; routes nothing calls). Check hardcoded localhost URLs, trailing slashes, wrong content types.	A B C	☐
4	Baseline tests	Backend `unittest`, frontend `tsc` and Vitest, production build, lint if configured, Playwright smoke, dependency audit, secret scan. Record real output and exit codes; separate old failures from new ones.	C + all	☐
5	Start the app safely	Section 10 commands. Confirm API base URL, database, env vars, static assets, understandable fallback when the AI key is missing.	All	☐
6	Click every control with Playwright	Section 7. Real user paths, accessible locators, control matrix with PASS / FAIL / BLOCKED / N-A, test both cancel and confirm for destructive actions.	A B	☐
7	Error detection and missing generated responses	Section 7 error list. Munim/AI: message is sent, backend receives it, provider is called (mock in automated tests), reply is non-empty and shown, failures are shown honestly, retries do not duplicate. Voice: permission, record, transcript, edit, failure paths.	A	☐
8	Authentication audit	Section 7, security matrix. Do not trust hidden routes; test the endpoint.	C	☐
9	Authorisation and data isolation	Use two synthetic businesses (two demo sandboxes) and the three roles: no credentials, wrong role, another business's IDs, changed IDs, protected fields (mass assignment), admin functions. Say plainly if a design limit exists.	C	☐
10	API and static endpoint audit	Per endpoint: methods, auth, missing/null/malformed/extra fields, types, payload limits, status codes, response shape. Static files: content type, caching, path traversal on the catch-all (`/..%2f`, encoded dots).	C	☐

6.4 Things that were not on any list (added)

#	Item	Why and what to check	Owner	✓
X1	Not-mounted code and duplicate APIs	See section 3. Decide delete or integrate; two backends in one repo is a bug-bounty gift (`/demo/reset` if ever mounted).	C	☐

X2	Branch drift	Diff `main`, `bugbounty`, `redundant` and the other branches (`fix`, `frontend`, `simulation-engine`, `multilingual-speech-to-text`). Document what only one branch has; make the three identical at the end.	A	☐
X3	Demo credentials and sandbox abuse	The README prints demo logins and the shared password. Production must use a different password; sandbox creation needs a cap per IP; 24-hour cleanup actually runs.	C	☐
X4	AI safety	Prompt injection via stored text (party notes, descriptions, setup text) reaching Munim; the number-grounding check rejects invented figures; no cross-business data in tool results; AI rate limit and size limit; behaviour with no key, slow provider, empty or malformed output.	A + C	☐
X5	Money and date correctness	Integer paise only; rounding; Indian grouping; negative amounts (`-₹`, never `₹-`); month-end, leap-day, IST midnight; horizon edges; zero and huge values; the pinned "today" never used in production.	A	☐
X6	Engine determinism and golden tests	Same books give the same run; same seed gives the same risk figures; the simulation contract fixtures in `fixtures/` and `docs/SIMULATION_*.md` still match.	A	☐
X7	Order undo and stock consistency	Every forward stage then back; purchase-order undo; no negative stock; invoice and payments agree; the "Books check" in Risk & Finance all pass after any action.	B	☐
X8	Concurrency and stale data	Two tabs, two users, the same record; a write while the risk job runs; the scenario cache (500 entries) and risk jobs do not grow without bound; double-click does not create duplicates.	C	☐
X9	Limits and pagination	Largest accepted sizes: body (413), list limits (transactions up to 1000), CSV size, chat length, number of parties/orders. Behaviour with 5,000 rows.	C	☐
X10	CSV and export safety	Formula injection (`=`, `+`, `-`, `@`), byte-order mark, non-Latin file names, large exports, role limits on who can export.	B	☐
X11	Languages	English, Hindi, Tamil screens and 10 voice languages; untranslated keys; text overflow in Tamil; date and number formats; fonts load.	B	☐
X12	Accessibility beyond contrast	Full keyboard use; focus order and visible focus; focus trapped in dialogs and returned on close; labels on every field; screen-reader names; reduced motion (Flow, pipeline, futures replay); 200% zoom.	B	☐
X13	Browser matrix	Chrome, Edge, Firefox, Safari (or WebKit in Playwright), a real phone. Voice input is Chromium-only by design: other browsers must hide or explain it.	B	☐
X14	Error boundary and offline	A crash inside one screen does not blank the app; API down, slow, offline, 401 mid-session (redirect to login without losing the page), 429 shows retry time.	A	☐
X15	Logging and privacy	No secrets, tokens, passwords, full request bodies or personal data in logs; log levels; one request ID per request; log size on the volume.	C	☐
X16	Dependencies and supply chain	`pip-audit` and `npm audit`; pinned versions; lockfiles committed; unused packages removed; licences; the Docker base image up to date.	C	☐
X17	Docker and Railway	Clean build from scratch; container runs as non-root; `/data` volume writable (a fix just landed on main); health check; restart keeps data; build does not bake in secrets.	C	☐
X18	Password and session policy	Minimum length, scrypt cost, change-password flow, same error for wrong user and wrong password, session lifetime, rotation after login, revoke-all works.	C	☐

X19	Documentation truth	README matches reality (routes, test counts, features, limitations); an env-variable table; run and deploy steps; a short runbook (what to do if the database file is lost).	A	☐
X20	Dead and noisy UI	Features nobody needs, duplicate boxes, placeholder text, "Lorem", test data in the UI, console logs, TODO text on screen. (Notebook p.3: the site felt redundant and overloaded.)	A + B	☐
X21	Print, share and message output	WhatsApp links and message drafts: encoding, length, placeholders filled, no business data leaking into URLs beyond what the user sends.	B	☐
X22	Legal and finance wording	Statutory references in the app (MSMED 45 days, s.269ST, s.40A(3)) are labelled "check with your CA"; investment ideas labelled indicative and not advice; GSTIN and Udyam checks are warnings where we cannot be sure.	A	☐

6.5 The new master prompt, phase by phase (Postman, Playwright, security, fixes)

This prompt supersedes the earlier one, which was cut off in section 10. It asks Claude Code to do the whole audit: inspect, inventory, run Postman and Playwright, fix, retest, report. Each person can run it scoped to their own track (see Appendix A). This table shows where each of its 12 phases is covered, so nothing in it is dropped.

Phase	What it asks for	Where it lands in this plan, and our additions	Owner	✓
1	Inspect the repository and keep the user's work safe: git state, unfinished merge or rebase, docs, stack, env, tests, Playwright and Postman files, Docker, CI. No reset, no force-push, no commit by Claude Code.	Section 2 and the kickoff step. Record the baseline (`git status`, branch, results of all existing tests) in your report before touching anything.	All	☐
2	Postman MCP and CLI: check connection and sign-in, extend or create `postman/collections/ripple-api.json`, an environment template, organised by area, documented per endpoint.	Section 7.4. Three separate states to report: MCP signed in or not; CLI installed and signed in or not; local collection run possible or not.	C + all	☐
3	Complete endpoint inventory in `docs/endpoint-inventory.json`; compare backend routes with every frontend request (wrong path, method, payload, base URL, trailing slash, placeholder data, static asset types, exposed debug routes, health route contents).	Q4 in 6.1 and X1. Add the unmounted-code verdict. The inventory records, per route: method, path, handler, auth, role, request fields, status, response shape, tables, frontend caller, test status.	C	☐
4	Postman tests with real assertions: the 16 cases, auth and authorisation by direct requests, safe data with read-back and clean-up, run through the CLI.	Section 7.4 and the security matrix 7.3. Cross-business tests use two sandbox businesses.	C + A B	☐
5	Playwright on every control, plus browser and network errors, error text on screen, and AI/chat and speech checks. Output: `docs/ui-control-coverage.json`.	Section 7.1 and 7.2, tracks A and B. Use the same PASS / FAIL / BLOCKED / NOT APPLICABLE statuses. Chat and voice are Chirantan's.	A B	☐

6	Security audit against OWASP Top 10, API Top 10, ASVS and WSTG: authentication, authorisation, input and injection, security configuration, secrets and dependencies.	N20 to N40, section 7.3, X3, X15 to X18. Secrets are never printed in reports: name the type and say rotate.	C	☐
7	Database and business logic: schema, integrity, rollback, duplicates, concurrency, rounding, time zones, stale cache; recompute selected financial scenarios independently and compare with the engine.	N32, X5, X6, X7, X8 and the new X27. Do not change a formula only to satisfy a test; agree the business rule first.	A + C	☐
8	AI and external services: missing or invalid key, outage, timeout, rate limit, malformed output, empty reply, history problems, prompt injection, unauthorised data in tool results, no keys in the frontend, no LLM-based authorisation or arithmetic.	X4 plus section 7.2 "missing replies". Test the failure path with mocks; never invent a successful reply.	A	☐
9	Files, performance and deployment: uploads, imports, exports, bounded performance checks, build, start commands, health checks, migrations, CI, rollback.	N33, X9, X10, X16, X17, X28. Bounded local tests only, never against the deployed site.	C + B	☐
10	Bug triage and fixes: severity Critical / High / Medium / Low, a full ledger entry, reproduce then root-cause then regression test then smallest fix, then rerun Postman and Playwright.	Section 9. Our scale P0 to P3 maps to Critical, High, Medium, Low. Ambiguous business rules are recorded as a decision for Chirantan, not guessed.	All	☐
11	Continuous verification: rerun Postman, backend, frontend, type-check, Playwright, full suite, build; review the diff; confirm no secrets or `node_modules` staged; check `git status`.	Cross-review step and final gate. Add a staged-files check before every pull request.	All	☐
12	Deliverables and an 18-point final report.	Return package, section 11, which now includes the Postman results and `ui-control-coverage.json`.	A assembles	☐

6.6 More items added by the new prompt

#	Item	What to check	Owner	✓
X23	Postman collection exists and runs	File committed, no secrets, runs through the CLI, every route has at least a valid case and a failure case, results recorded.	C	☐
X24	Machine-readable coverage files	`docs/endpoint-inventory.json` and `docs/ui-control-coverage.json` exist and agree with the matrices.	A C	☐

X25	SSRF, open redirect, command injection, path traversal, header injection, template injection, unsafe deserialisation	For this app: find any place that fetches a URL, redirects, builds a file path, runs a command, reads a header into a response, renders a template from user text (message drafts) or loads data with `pickle` / `eval`. If none exists, record "not applicable, because ...".	C	☐
X26	Retry and duplicate safety	Double-clicks, repeated POSTs and network retries must not create duplicate payments, orders, invoices or chat actions.	C + B	☐
X27	Independent financial recomputation	Pick at least five synthetic scenarios (late customer, split supplier payment, extension, a new spend, an order undo). Calculate cash, net position and the first shortfall by hand or in a spreadsheet; compare with the engine and with Risk & Finance figures.	A	☐
X28	Repository state and staging hygiene	Before work: `git status`, branch, no merge or rebase in progress. Before every pull request: nothing from `node_modules`, `dist`, databases, `.env`, test results or screenshots with real data is staged.	All	☐
X29	Health and info exposure	`/health` and `/api/health` reveal nothing beyond "ok" and a version at most. No debug or admin route is reachable without a session.	C	☐
X30	Voice and speech-to-text paths	Microphone permission denied, no speech heard, unsupported browser, service error, and an edited transcript sent. Note which part runs in the browser and which on our server.	A	☐

7. How to test (the method)

7.1 Click every control (tracks A and B)

1. For each screen, list every interactive control in a **control matrix** (CSV or table): route, control name and role, preconditions, action, expected result, actual result, status (PASS, FAIL, BLOCKED, N/A), evidence, regression test.
2. Open each route both directly (typing the address) and through navigation. Reload the page. Open it in a second tab.
3. Use accessible locators (`getByRole`, `getByLabel`, `getByPlaceholder`) rather than CSS paths. A control that cannot be found by role or label is itself an accessibility bug.
4. A click only counts as PASS if the request was sent, the response was right, the screen shows the new state, the state survives a reload, a second click does not duplicate it, and the console is clean. "It was clickable" is not a pass.
5. Reach hidden states: empty data, lots of data, error state, loading state, each tab, each role, a selected row, an open dialog, a long name, Tamil text.
6. Destructive actions (delete material, reset data, undo order stage, remove team member): test Cancel, then confirm, on synthetic data only. External actions (WhatsApp links, AI provider) are mocked.

7.2 What to capture while you click

Class	Flag it when you see
Browser	`pageerror`, unhandled promise rejection, console error or warning that points to a defect, React render error, missing script/style/image/font, failed source map.
Network	Unexpected 400, 401, 403, 404, 405, 409, 422, 429, 500, 502, 503; failed or reset requests; wrong payload; wrong or missing `Content-Type`; redirect surprises; stale or hardcoded URLs; a request pending forever; a 200 with an empty or malformed body. A deliberate 422 for invalid input is correct: say what the expected result was.
Text on screen	`Internal Server Error`, `Traceback`, a stack trace, raw SQL or provider errors, `undefined`, `null`, `[object Object]`, raw JSON, "Request failed", a success toast after a failed request, a blank or endlessly loading screen, internal identifiers shown as text.
Missing replies	A button that sends a request and never produces a visible result is a failure, even without an exception. Applies above all to Munim chat, setup draft, what-if, payment check, datasheet download and message drafts.

7.3 Security matrix (track C)

Test	For every sensitive endpoint
Authentication	No cookie; garbage cookie; expired session; logged-out session replayed; password wrong / empty / very long; same message for unknown user; lockout and 429 after repeated attempts; session after "log out everywhere"; two tabs.
Function-level authorisation	3 roles x all 73+ routes. Expected result is written first (allowed / 403), then tested at the API, not through the UI.
Object-level authorisation	Business A's session asks for business B's party, invoice, order, material, transaction, run, scenario key, draft, export. Change IDs in paths and bodies. Expect 404 or 403, never data.
Property-level (mass assignment)	Send extra fields on create and update: `business_id`, `role`, `is_sandbox`, `data_version`, amounts on a supervisor's call, status fields. They must be ignored or rejected.
Input handling	Missing, null, wrong type, extra, huge, negative, zero, NaN-like, Unicode, control characters, nested JSON, duplicate keys; body just over the limit; wrong `Content-Type`; wrong method (405).
Injection and XSS	SQL strings in every filter and ID; HTML and script payloads in every stored text; CSV formula cells; template-like text in message drafts and chat; check each render site.

Static files	Path traversal against the catch-all route, directory listing, source maps, `.env`, `.git`, database file paths, backup files; cache headers on assets versus pages.
Transport and headers	HTTP to HTTPS redirect, HSTS, cookie flags, CSP effect, CORS from another origin, CSRF without the header, clickjacking in a frame.
Abuse	Rate limits on login, demo creation, AI, setup draft; behaviour of `Retry-After`; resource limits on scenarios and risk jobs. Gentle local checks only.
Information leakage	Error bodies, headers (server and version), logs, response fields that should not leave the server (password hash, internal IDs, other users' emails).

7.4 Postman API testing (new master prompt, phases 2 to 4)

Postman tests the backend directly, without the browser. It does not replace Playwright. Each person checks their *own* Claude Code session: the Postman MCP connection, the Postman CLI and local collection runs are three separate things, and signing in to one does not sign in to the others.

Step	What to do (and what not to do)
Check the MCP	Ask Claude Code whether the Postman MCP server is connected and signed in. If it needs authentication, sign in through the connector settings yourself; **never paste passwords, API keys or tokens into the chat**. Until it is signed in, nobody may claim a remote workspace or collection was inspected. Carry on with local testing.
Check the CLI	`postman --version`, then the login state (without printing any key). If it is missing, install it only if your environment allows. Local runs of a collection file may work without remote login.
Where the collection lives	`postman/collections/ripple-api.json` plus `postman/environments/local.example.json` (no real values). Extend the file if it already exists; never delete unrelated requests. If a shared remote collection cannot be identified with confidence, build it locally and document how to import it.
Ripple specifics the collection must handle	All routes live under `/api`. Sign-in is `POST /api/auth/login` (or `POST /api/auth/demo` for a private sandbox) and sets an **HttpOnly session cookie**, so the collection needs a cookie jar. Every POST, PUT, PATCH and DELETE needs the header `X-Ripple: 1` or the server answers 403. Money is whole **paise** integers, dates are ISO. Errors come back as `{"error": {"code": ..., "details": ...}}`. Login, demo and AI routes are rate limited, so space the requests and expect 429 to be a valid result in the abuse tests.
Who owns which folder	**John:** health, auth, session, team, data reset/load, settings, export, security cases. **Chirantan:** engine, scenarios, fixes, risk, guard (risk and finance), Munim chat, book, today. **Nandini:** parties, invoices, payments, orders, inventory, materials, recipes, datasheets, setup and import. One collection file, one pull request per folder, so nobody overwrites another's work.
Assertions, not status codes	For each request check: status, content type, required fields and types, the error schema on failures, and for every write a follow-up read that proves it was stored. 200 alone is not a pass.
The 16 cases per endpoint	Valid; missing required field; empty; wrong type; malformed value; null where forbidden; extra field; invalid ID; nonexistent record; no credentials; wrong role; duplicate request; dependency failure; wrong content type; oversized body (safe size); malformed response handling. An expected 400, 401, 403, 404, 409, 422 or 429 is a pass when the test was designed to provoke it.
Safe data	Create synthetic records, read them back, update, test invalid operations, then delete only what the run created. Use a sandbox business (`/api/auth/demo`), never the shared demo or real data. No real payments, offers or messages.
Run and record	`postman collection run ./postman/collections/ripple-api.json` with the environment option your installed CLI supports (check `postman collection run --help`; do not copy a command blindly). Record total requests, passed and failed assertions, unexpected statuses, request failures, auth failures, persistence failures, blocked tests. If the CLI cannot run, say exactly why. Never invent CLI output.

Secrets	No API key or token in the collection, environment, report, screenshot or git history. Use the environment template with empty values, and supply real values locally through the Postman app or your shell.

One conflict to avoid. The master prompt writes to single shared files (`docs/production-readiness-audit.md`, `docs/bug-ledger.md`, `docs/endpoint-inventory.json`, `docs/ui-control-coverage.json`). If three people run it in the same files, you will overwrite each other. Each person writes to `docs/audit/<name>/` with the same file names; Chirantan merges them into the shared files at the end. The prompt also says "do not commit or push": that stays true for Claude Code. You commit and push your own branch by hand under the branch rules in section 2.

8. Hardcoded versus real: starter list for notebook Q1 to Q3

These are the constants and static content we already know about. Extend this by search. For each, choose: **setting**, **keep as constant**, **demo only**, or **label as assumption**.

Where	Hardcoded value	Proposal
`services/fixes.py`	Early-payment discount 2%, invoice-finance advance 90% and fee 1.5% per 30 days, supplier extension 10 days. These are invented terms shown as "ways to fix".	Make them business settings with defaults, and show "assumed terms" next to every fix. (M)
`services/risk.py`, `position.py`	Shrinkage k=5, minimum 20 payments, windows 30/60/90 days, 200 samples.	Keep as documented model constants; show them in the "how far to trust it" panel. (S)
`services/guard.py`	Concentration 40%/60%, overdue 20%/40%, "slipping" +5 days, 45-day MSME limit, cash rules ₹2,00,000 and ₹10,000, outlier 3x, duplicate window 3 days, offer minimum ₹25,000, default yield 6.5%, 30-day payment window.	Statutory ones stay constants with a citation. The thresholds become settings. Default yield shown as an editable assumption (already is). (M)
`records/seed.py`	The demo business "Mahalakshmi Fabrics", synthetic payment history, 8 customers, 6 suppliers, sample orders and stock.	Demo only. Make sure no screen or document presents it as real; keep "demo copy" label visible. (S)
Demo accounts	Three shared logins and a documented password.	Production password from environment only, remove it from the public README before any public deployment. (S)
`services/messages.py`, `lib/i18n.ts`	English, Tamil, Hindi message templates and all screen text.	Static by design. Check for untranslated keys. Other languages exist for voice only. (S)
Settings defaults	Horizon 90 days, safety buffer, customer and supplier terms 30 days.	Fine as defaults; confirm they are editable and validated. (S)
Frontend	Page titles, button text, help text, the legend and caption text, the unused Today boxes (already removed).	Static text: fine. List remaining placeholder or filler text. (S)
Everything calculated	Cash, net position, runway, shortfall day, chance, safe-to-spend, fixes' effects, flags, checks.	These come from the engine or records and must never be typed in: confirm by test.

Real data that could replace synthetic data: import the owner's bank statement or Tally export as CSV; import parties and invoices; real GSTIN validation against the checksum (already a warning); WhatsApp message sending through an approved provider (today it is a share link only).

9. Bug handling

Severity	Meaning	Rule
P0	Data leak or cross-business access, authentication bypass, remote code or injection, data loss, app does not start or crash on login.	Stop everything, tell the others, fix first.
P1	A core flow is broken or wrong: wrong money, wrong forecast, a button that does nothing, a failed request shown as success, 500 errors.	Fix in this round.
P2	Wrong but with a workaround: layout clipping, confusing text, missing validation message, minor accessibility failures.	Fix if time allows; otherwise list.
P3	Cosmetic or an idea.	List only.

Ledger entry (one per bug), IDs like `A-007`, `B-012`, `C-003` (track letter + number):

```
ID | Severity | Branch(es) | Where (route / endpoint / file)
Steps to reproduce (numbered, with the exact data used)
Expected | Actual | Evidence (screenshot, console text, response, sanitised)
Root cause | Fix (commit id) | Regression test (file::name) | Verified by (name)
Status: OPEN / FIXED / VERIFIED / WONT-FIX (reason)
```

Fix workflow: reproduce, write the failing test, fix, test passes, run the whole suite, pull request, a colleague re-tests it, tick the ledger. Fix the root cause, not the symptom. If a fix touches another track's files, tell the owner first.

10. Setup and commands (Windows)

1. Copy or clone the repo to a path **without** `#`, for example `C:\ripple` (Vite and Vitest fail inside "M# 2026").
2. Backend: `python -m venv .venv`, activate, `pip install -r requirements.txt` (and `requirements-voice.txt` if you test voice).
3. Frontend: `cd frontend`, `npm ci`, `npx tsc -b`, `npm run build`.
4. Run: from the repo root, with `DATABASE_URL=:memory:` for throw-away data and `FRONTEND_DIST=frontend\dist`: `python -m uvicorn backend.app.main:app --port 8765`. For the deterministic demo date use `RIPPLE_TODAY=2026-10-14` (testing only, never on a real deployment).
5. Backend tests: `python -m unittest discover -s tests -p "test_*.py"` (expect 127 passing, 1 skipped). Frontend unit tests: `npx vitest run` in the clean-path copy.
6. Test accounts: the demo accounts in the README, or the "Open demo business" button, which creates a private sandbox (best for isolation and for two-business tests: click it twice in two browser profiles).
7. Playwright: `e2e/smoke.mjs` already exists (set `RIPPLE_URL` and `BROWSER_PATH`). Add new specs as Playwright Test files and keep traces and screenshots in an ignored folder such as `test-results/`.
8. Postman: `postman --version`; run the collection with `postman collection run ./postman/collections/ripple-api.json` plus the environment option of your installed version (see section 7.4).
9. Scans: `pip-audit`, `npm audit --omit=dev`, `gitleaks detect --log-opts="--all"` (or trufflehog), Lighthouse in Chrome DevTools.

Starter for the endpoint comparison (John): list route decorators with `grep -rn "@router\.\(get\|post\|put\|patch\|delete\)" backend/app`, list frontend calls with `grep -rn "api\.\(get\|post\|put\|patch\|delete\)\|fetch(" frontend/src`, and diff the two lists by path.

> **Capture this error-listener in every Playwright spec** so nothing slips through: page `console` (error and warning), `pageerror`, `requestfailed`, and `response` with status 400 or above on `/api/`. Fail the test on any of them unless the test declares the expected one.

11. Return package: what each of you sends back

Send Chirantan, in one folder or one message, by the end of the report phase:

1. **Code:** the name of your pushed branch (or a patch) with all fixes, based on the latest `main`, with tests passing.
2. **Report** (template below), one page to three pages.
3. **Bug ledger** (every bug, including unfixed ones).
4. **Control matrix** for your screens (track A, B), also as `ui-control-coverage.json`; or **endpoint inventory** and **security matrix** (track C).
5. **Postman:** your folder of `postman/collections/ripple-api.json`, the CLI run summary (requests, passed and failed assertions, blockers), and the MCP and CLI status (connected or not, signed in or not).
6. **The 18-point final report** from the master prompt, if you ran it (Appendix A).
7. **Evidence folder:** screenshots, traces, command output (secrets removed).

> **Report template (copy this):**

```
NAME / TRACK / BRANCH / COMMIT RANGE
1. Scope covered (routes, endpoints, checklist IDs) and NOT covered (and why)
2. Test results: backend N pass / M fail; frontend N pass / M fail; build; Playwright N pass / M fail
3. Bugs: found P0 _ P1 _ P2 _ P3 _ | fixed _ | open _  (list IDs, one line each)
4. Checklist: ticked IDs, and for every unticked ID the reason
5. Answers to notebook Q1 to Q6 (only the ones in your track)
6. Security / privacy findings (private, not to be pasted anywhere public)
7. Risks still open, and what you would do next
```

```
8. Things you changed that others must know (migrations, env vars, new routes, removed code)
9. Branch state: which of main / bugbounty / redundant contain your fixes
```

Final gate (all must be true before we call it done)

☐	Zero open P0 and P1 bugs. Every P2 and P3 either fixed or listed with a reason.
☐	Backend, frontend, build and Playwright suites are green on the merged `main`; every fixed bug has a regression test.
☐	Every control on every screen is in a matrix with a status; no BLOCKED without a reason.
☐	Endpoint inventory complete; unwanted endpoints removed or justified; frontend and backend lists match.
☐	Security matrix passed: auth, roles, cross-business access, mass assignment, injection, XSS, CSRF, CORS, headers, cookies, rate limits.
☐	No secrets in the repository history, the bundle, logs or reports. Any exposed key rotated.
☐	The Postman collection runs through the CLI with all assertions passing (or each failure is a logged bug); every endpoint has a valid and a failure case.
☐	Notebook Q1 to Q6 answered; N1 to N40 and X1 to X30 ticked or explained.
☐	`main`, `bugbounty`, `redundant` hold the same final commit (synced by Chirantan, outside the freeze).
☐	README and docs match the product.

An honest note: nobody can prove every bug is found. The aim is to cover every route, endpoint, control and important state we can reach, fix what we find, and say clearly what we could not test.

Appendix A: Running the master prompt in Claude Code, per person

Chirantan has the full master prompt text (the long "Ripple / HitNRun-4 Production Audit, Postman API Testing..." prompt). Each of you pastes it into your own Claude Code session, with a short **scope header on top** so three sessions do not collide. Use the header for your track:

```
SCOPE FOR THIS RUN (read first, it overrides conflicting instructions below)
- I am Track [A | B | C] ([Chirantan | Nandini | John]).
- Test and fix ONLY: [paste your scope from section 4].
- Other tracks' areas: report problems you notice, do not fix them.
- Write all deliverables to docs/audit/[name]/ (same file names as in the prompt),
  not to the shared docs/ files. Do not overwrite anyone else's files.
- Work on a branch fix/[initials]-[topic] cut from the latest main.
- Do not commit or push. I will review the diff and commit myself.
- Never print or store secrets. Use sandbox businesses and synthetic data only.
```

Before you start, make sure: the repo is on a path without `#`, the backend runs, `postman --version` answers, and Playwright can open Chrome or Edge. Tell your Claude Code session which of these are not ready so it reports them as blockers instead of guessing.

> **What "done" looks like for the prompt's 18-point final report:** routes and controls counted with PASS / FAIL / BLOCKED / N-A; endpoints counted and tested; Postman MCP status; Postman CLI status; collection path and run results; passed and failed assertions; browser errors and error text found; authentication and authorisation findings; security findings by severity; AI, chat and voice findings; financial and database findings; bugs fixed and files changed; regression tests and results; real build and type-check results; unresolved defects; blocked areas; a justified release recommendation. Paste each person's report back into the working chat and Chirantan merges them.